WHAT IS SYCL?

LEARNING OBJECTIVES

- Learn about the SYCL specification and its implementations
- Learn about the components of a SYCL implementation
- Learn about how a SYCL source file is compiled
- Learn where to find useful resources for SYCL

WHAT IS SYCL?

SYCL is a single source, high-level, standard C++ programming model, that can target a range of heterogeneous platforms

WHAT IS SYCL?

A first example of SYCL code. Elements will be explained in coming sections!

```
#include <sycl/sycl.hpp>

int main(int argc, char *argv[]) {
  std::vector<float> dA{2.3}, dB{3.2}, dO{7.9};

  try {
    auto asyncHandler = [&](sycl::exception_list eL) {
      for (auto &e : eL)
        std::rethrow_exception(e);
    };
    sycl::queue gpuQueue{sycl::default_selector{}, asyncHandler};

    sycl::buffer bufA{dA.data(), sycl::range{dA.size()}};
    sycl::buffer bufB{dB.data(), sycl::range{dB.size()}};
    sycl::buffer bufO{dO.data(), sycl::range{dO.size()}};

    gpuQueue.submit([&](sycl::handler &cgh) {
      sycl::accessor inA(bufA, cgh, sycl::read_only);
      sycl::accessor inB(bufB, cgh, sycl::read_only);
      sycl::accessor out(bufO, cgh, sycl::write_only);

      cgh.parallel_for(sycl::range{dA.size()},
                       [=](sycl::id<1> i) { out[i] = inA[i] + inB[i]; });
    });

    gpuQueue.wait_and_throw();

  } catch (sycl::exception &e) {
                YCL exception */

}
```

Managing the data

Work unit

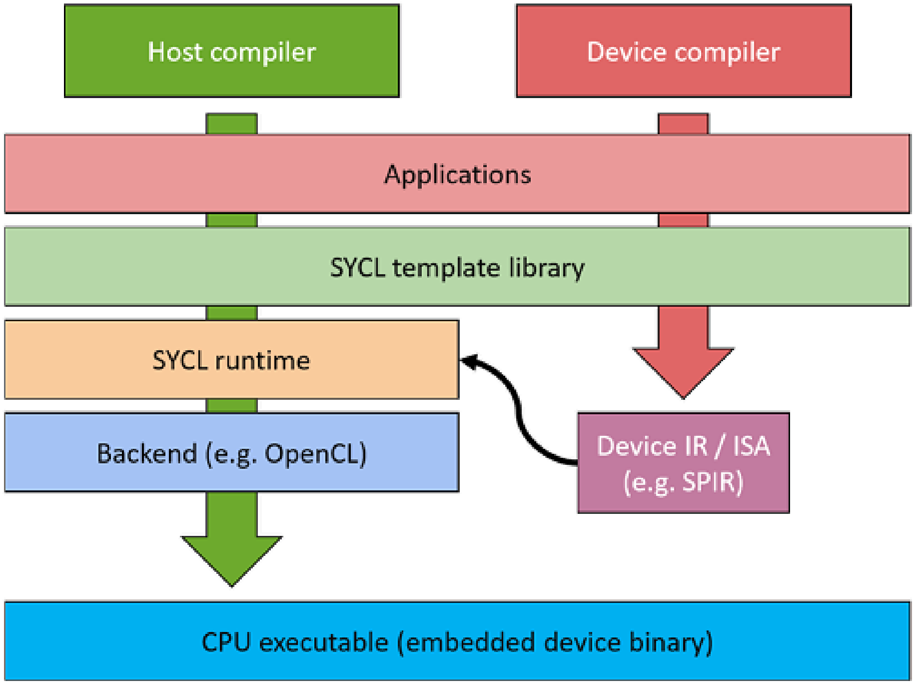
Device code

SYCL IS...

- SYCL extends C++ in two key ways:
  - device discovery (and information)
  - device control (kernels of work, memory)
- SYCL is modern C++
- SYCL is open, multivendor, multiarchitecture

WHAT IS SYCL?

SYCL is a ***single source***, high-level, standard C++ programming model, that can target a range of heterogeneous platforms



- SYCL allows you to write both host CPU and device code in the same C++ source file
- This requires two compilation passes; one for the host code and one for the device code

NHR SW

WHAT IS SYCL?

SYCL is a single source, ***high-level***, standard C++ programming model, that can target a range of heterogeneous platforms

- SYCL provides high-level abstractions over common boilerplate code
  - Platform/device selection
  - Buffer creation and data movement
  - Kernel function compilation
  - Dependency management and scheduling

WHAT IS SYCL?

SYCL is a single source, high-level ***standard C++*** programming model, that can target a range of heterogeneous platforms

```
array_view<float> a, b, c;
<2> idx) restrict(amp) {
```

```
std::vector<float> a, b, c;

#pragma parallel_for
for(int i = 0; i < a.size(); i++) {
  c[i
}
```

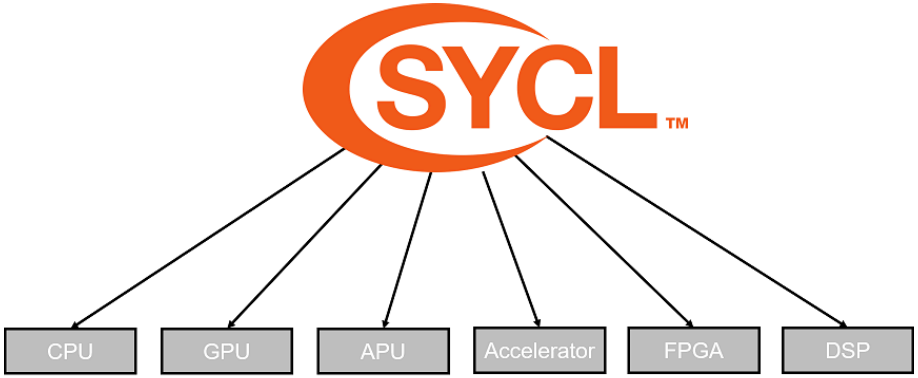
```
__global__ vec_add(float *a, float *b, float *c) {
  return c[i] = a[i] + b[i];
}

float *a, *b, *c;
vec_add<<<range>>>(a, b, c);
```

```
cgh.parallel_for(range, [=](cl::sycl::id<2> idx) {
  c[idx] = a[idx] + b[idx];
});
```

- SYCL allows you to write standard C++
  - SYCL 2020 is based on C++17
- Unlike the other implementations shown on the left there are:
  - No language extensions
  - No pragmas
  - No attributes

WHAT IS SYCL?

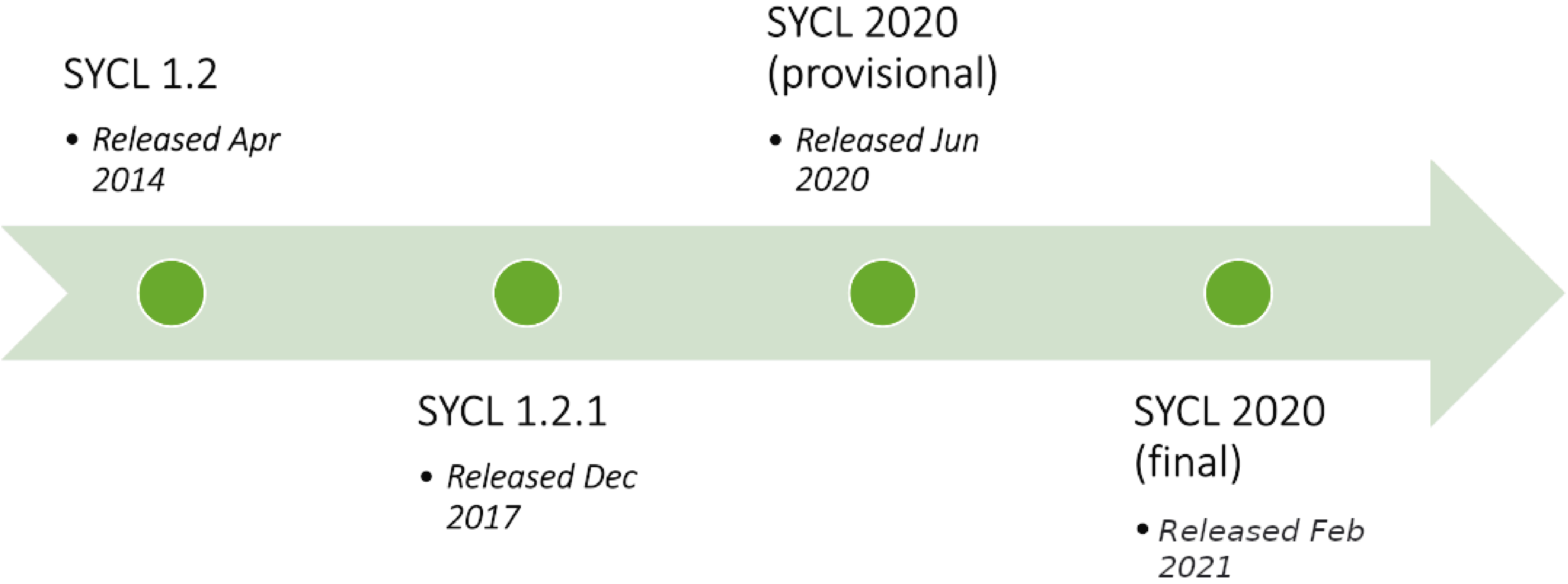
SYCL is a single source, high-level standard C++ programming model, that can **target a range of heterogeneous platforms**



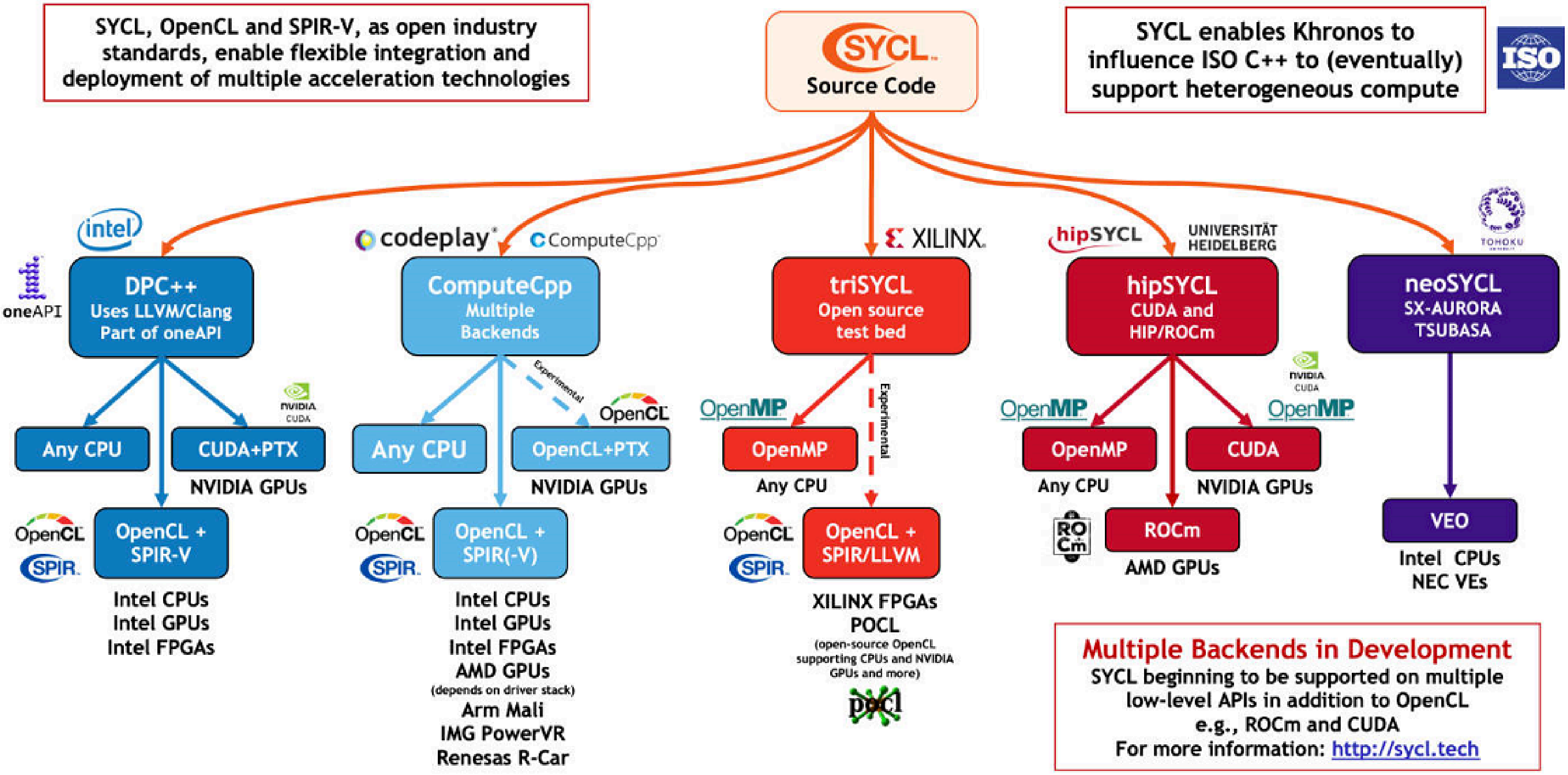
- SYCL can target any device supported by its backend
- SYCL can target a number of different backends

SYCL has been designed to be implemented on top of a variety of backends. Current implementations support backends such as OpenCL, CUDA, HIP, OpenMP and others.

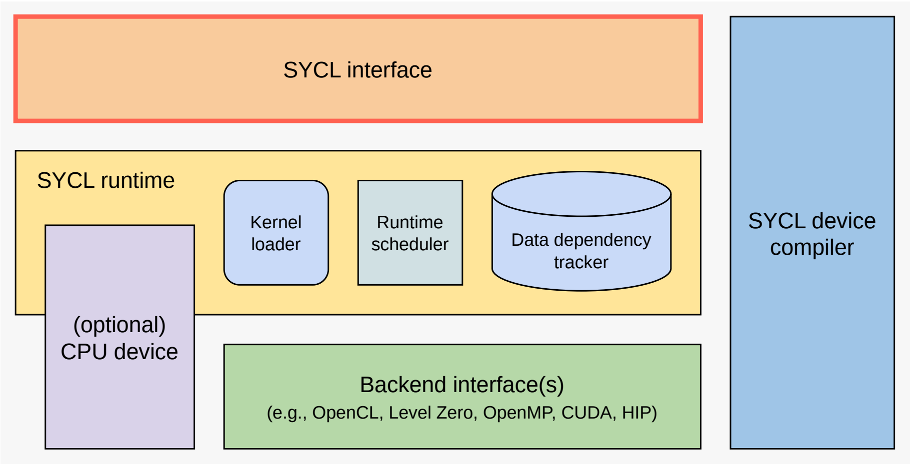
SYCL SPECIFICATION



SYCL IMPLEMENTATIONS



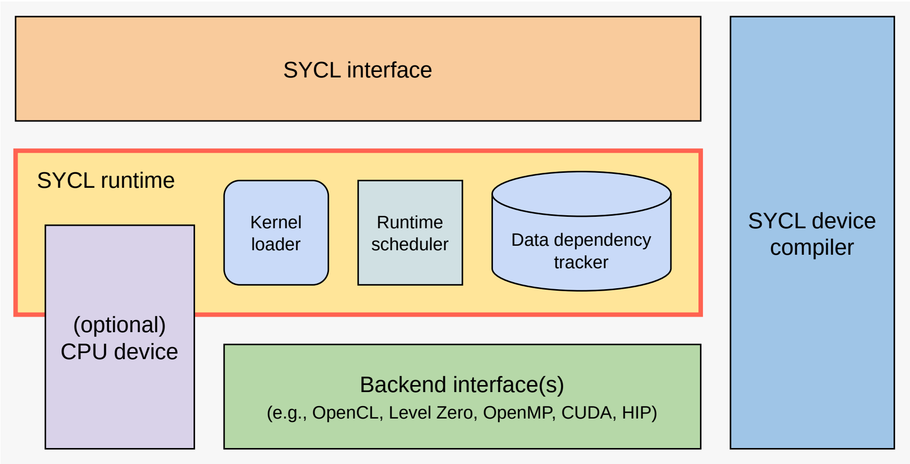
WHAT A SYCL IMPLEMENTATION LOOKS LIKE



- The SYCL interface is a C++ template library that developers can use to access the features of SYCL
- The same interface is used for both the host and device code

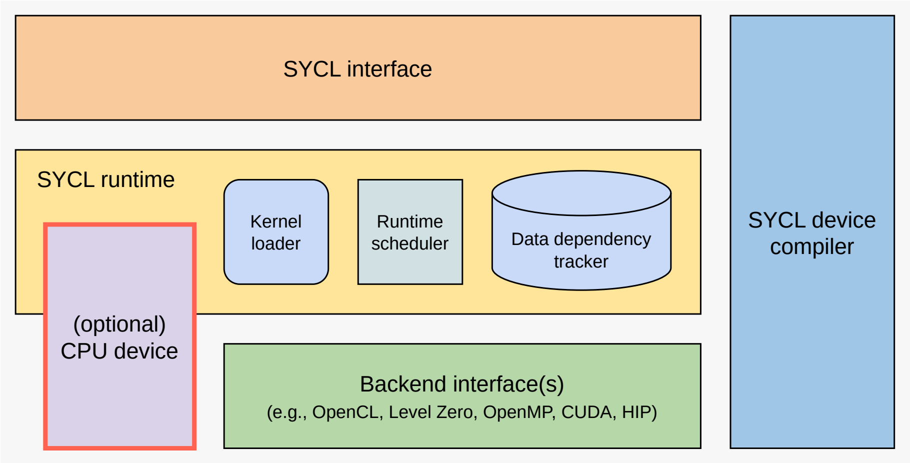
- The host is generally the CPU and is used to dispatch the parallel execution of kernels
- The device is the parallel unit used to execute the kernels, such as a GPU

WHAT A SYCL IMPLEMENTATION LOOKS LIKE



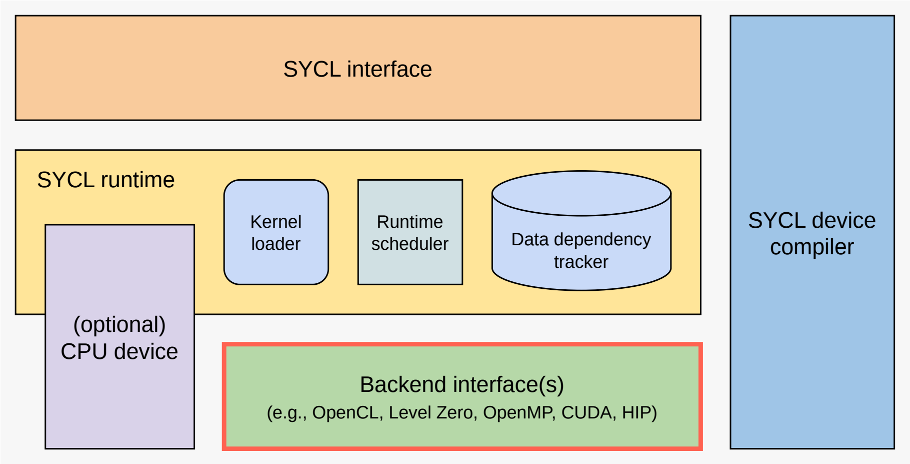
- The SYCL runtime is a library that schedules and executes work
  - It loads kernels, tracks data dependencies and schedules commands

WHAT A SYCL IMPLEMENTATION LOOKS LIKE



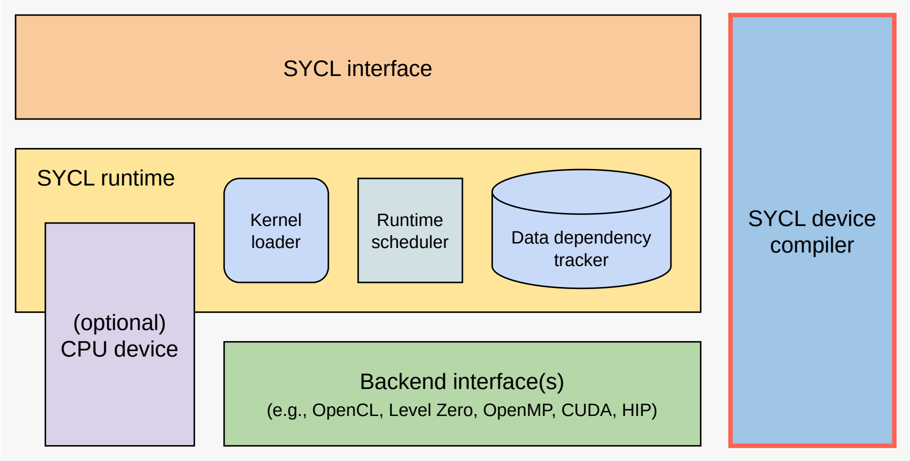
- There is no Host Device in SYCL (as of SYCL 2020)
- SYCL 1.2.1 had a concept of a 'magical' host device - an emulated backend
- SYCL 2020 implementations generally offer a CPU device
- Often, the best debugging on a platform is using a CPU device
- Yet, debugging off the CPU is important to discover offloading issues

WHAT A SYCL IMPLEMENTATION LOOKS LIKE



- The back-end interface is where the SYCL runtime calls down into a back-end in order to execute on a particular device
- Many implementations provide OpenCL backends, but some provide additional or different backends.

WHAT A SYCL IMPLEMENTATION LOOKS LIKE

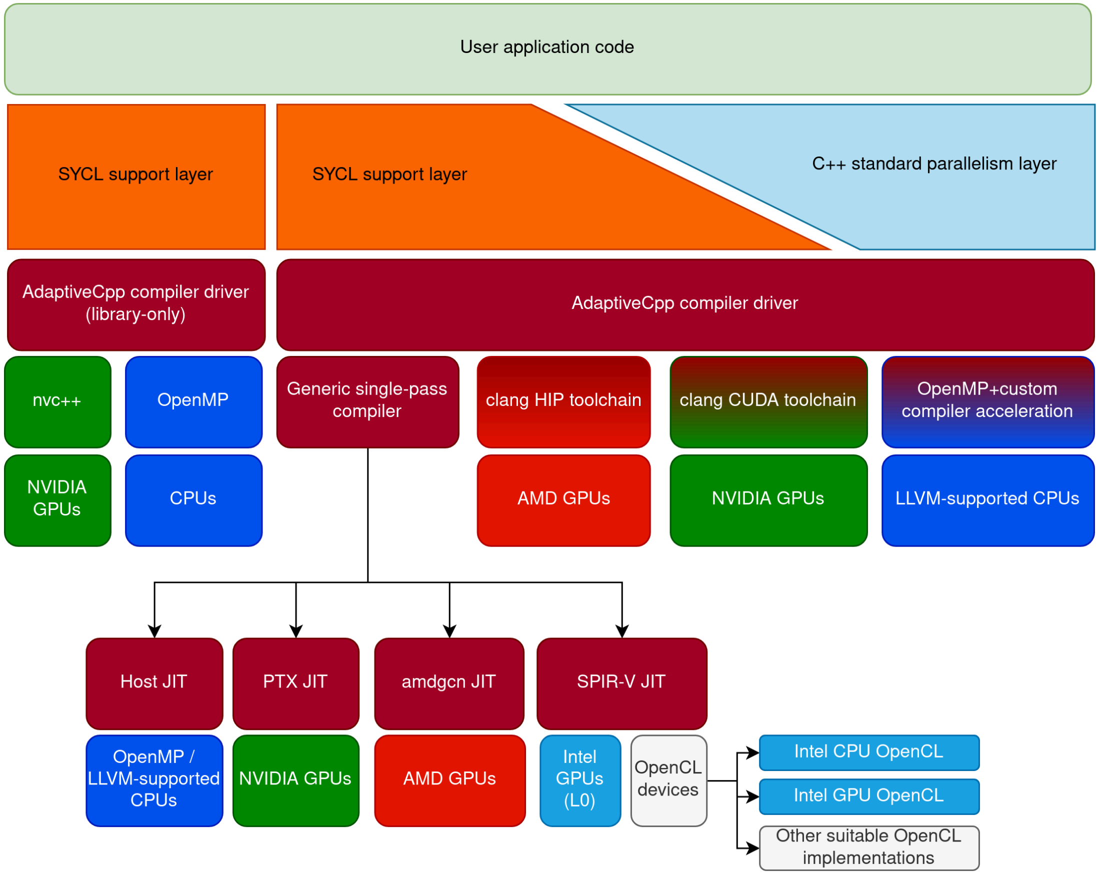


- The SYCL device compiler is a C++ compiler which can identify SYCL kernels and compile them down to an IR or ISA
    - This can be SPIR, SPIR-V, GCN, PTX or any proprietary vendor ISA
- Some SYCL implementations are library only in which case they do not require a device compiler

IR = Intermediate Representation **ISA** = Instruction Set Architecture

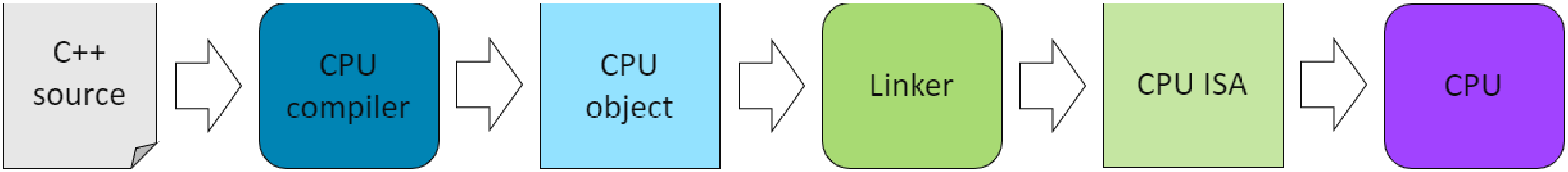
NHR SW

WHAT ADAPTIVECPP LOOKS LIKE



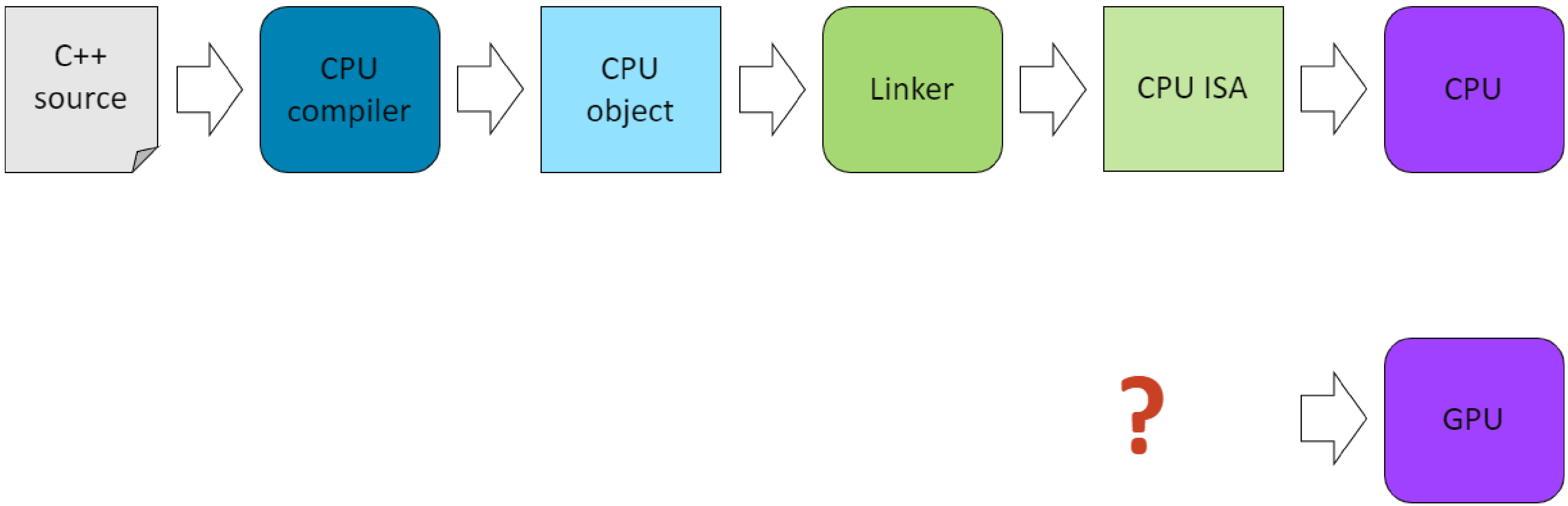
- AdaptiveCpp is a multi-backend SYCL implementation
    - It supports NVIDIA, AMD, Intel GPUs + any CPU
- Additionally, it implements C++ standard parallel algorithms on all those devices

STD C++ COMPILATION MODEL



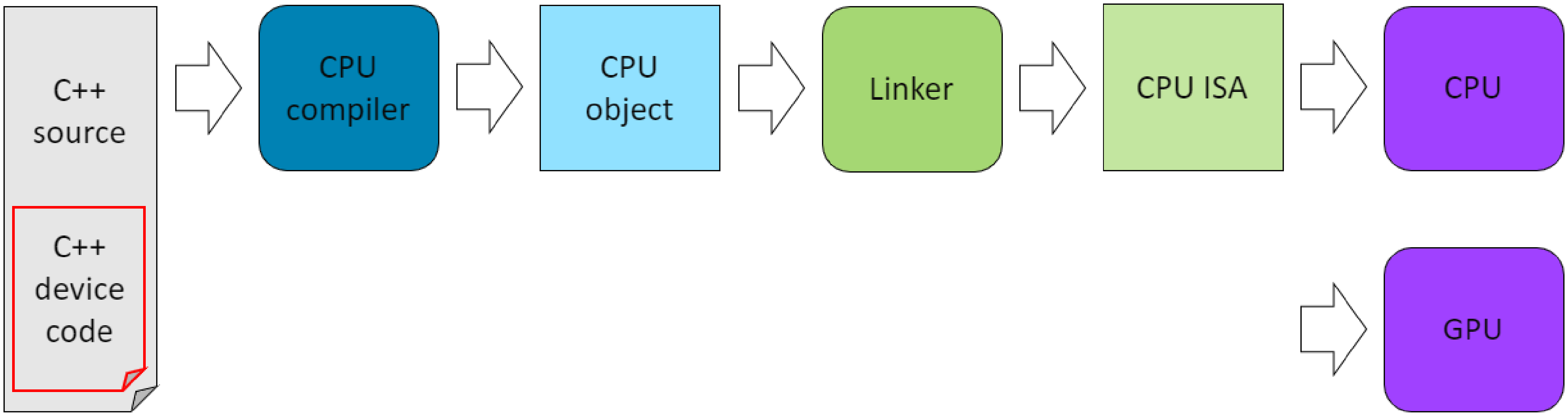
- This is the typical compilation model for a C++ source file.

STD C++ COMPILATION MODEL



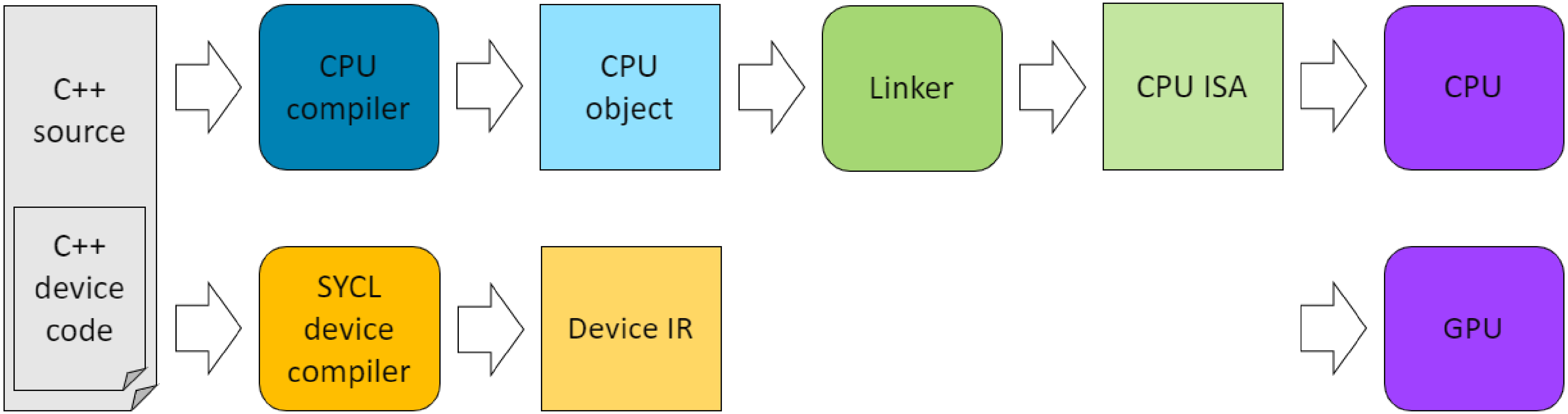
- So how do you compile a source file to also target the GPU?

STD C++ COMPILATION MODEL



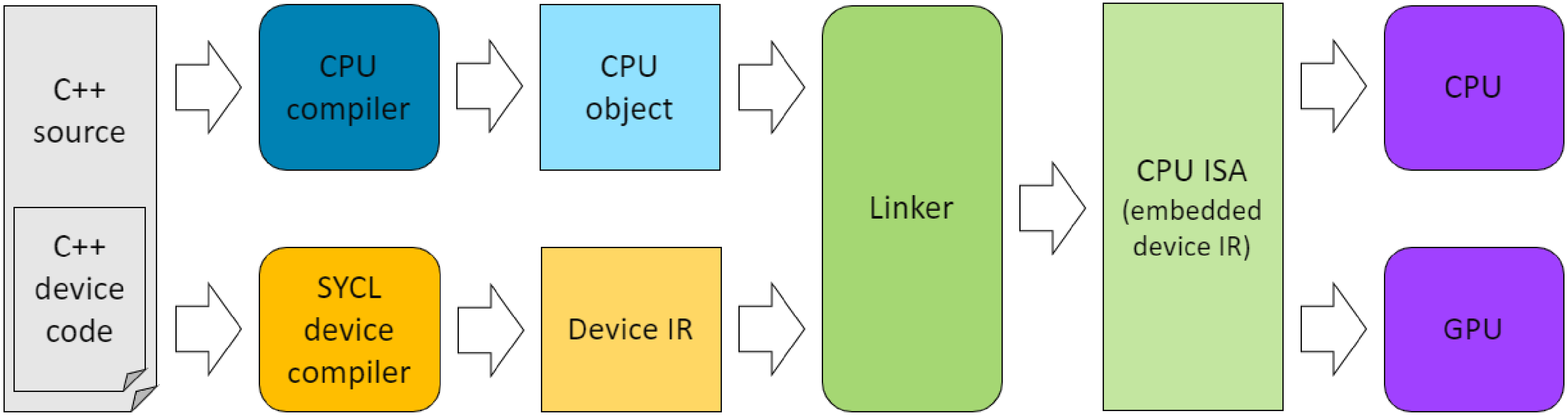
- As SYCL is single source the kernel functions are standard C++ function objects or lambda expressions.
- These are defined by submitting them to specific APIs.

STD C++ COMPILATION MODEL



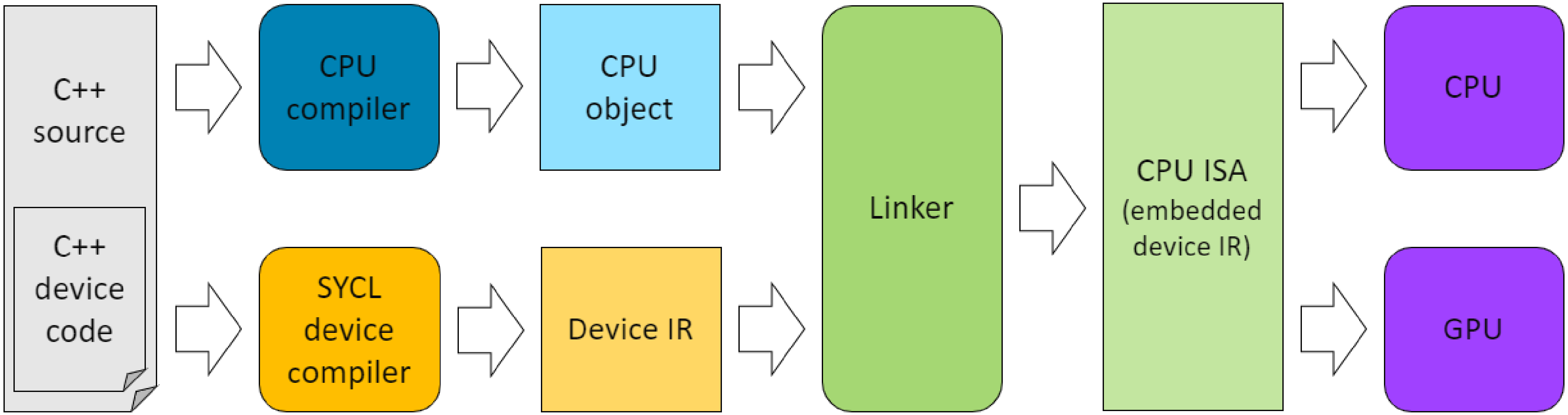
- As well as the standard C++ compiler, the source file is also compiled by a SYCL device compiler.
- This produces a device IR such as SPIR, SPIR-V or PTX or ISA for a specific architecture containing the GPU code.

STD C++ COMPILATION MODEL



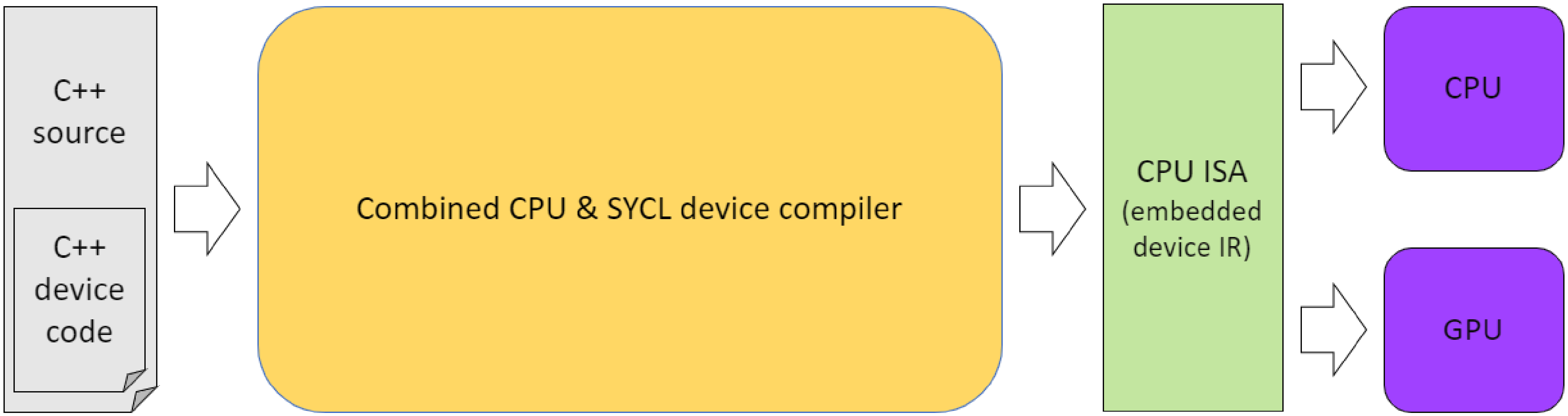
- The CPU object is then linked with the device IR or ISA to form a single executable with both the CPU and GPU code.

STD C++ COMPILATION MODEL



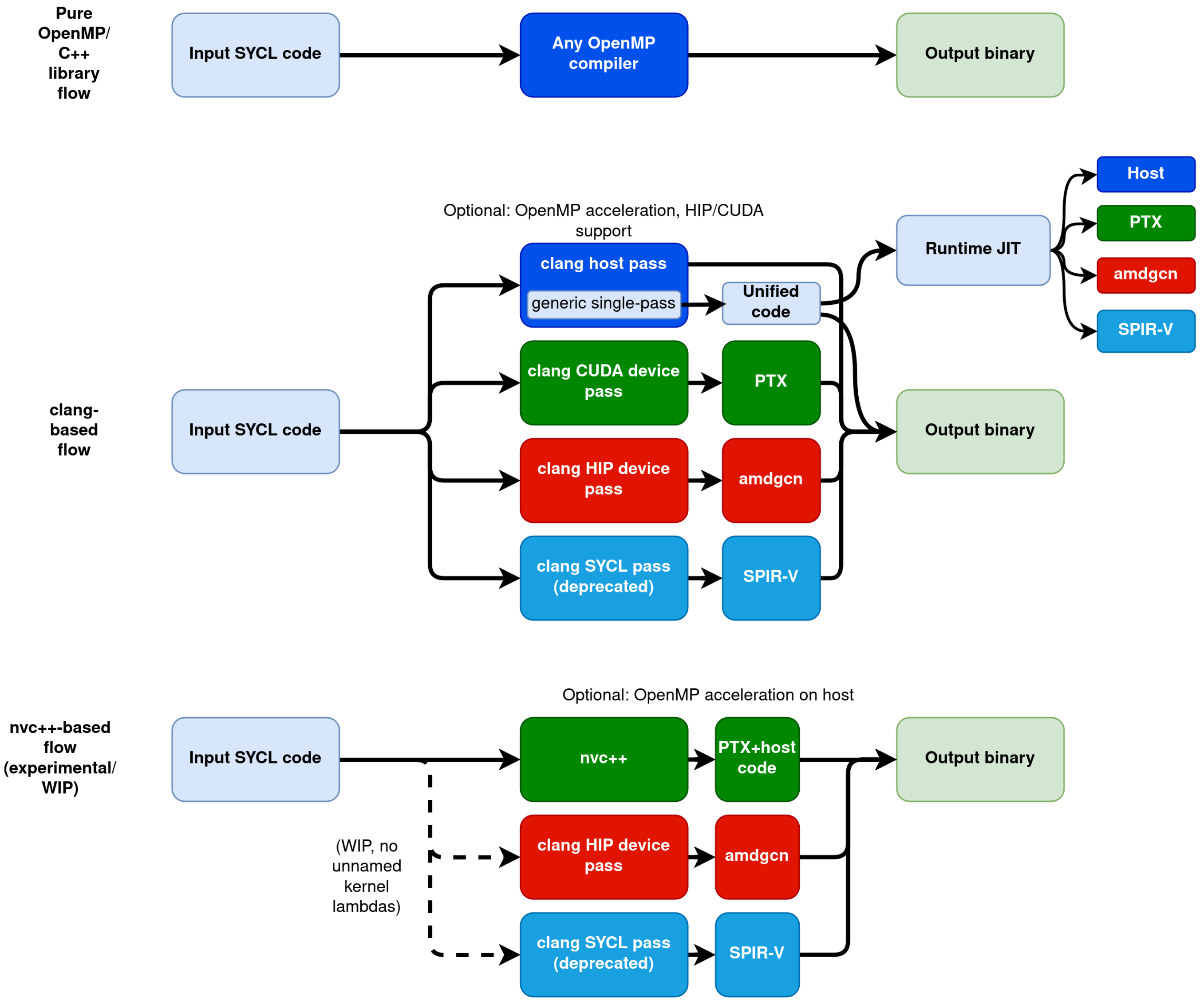
- This is the multi-compiler compilation model.
- This allows the host compiler (MSVC, clang, icx, gcc) to be independent of the SYCL device compiler.

STD C++ COMPILATION MODEL



- SYCL also supports a single-compiler compilation model.
- Where both the host compiler and SYCL device compiler are invoked from the same driver.

ADAPTIVECPP COMPILATION MODELS



WHERE TO GET STARTED WITH SYCL

- Visit **https://sycl.tech** to find out about all the SYCL book, implementations, tutorials, news, and videos
- Visit **https://www.khronos.org/sycl/** to find the latest SYCL specifications
- Checkout the documentation provided with one of the SYCL implementations.

QUESTIONS

EXERCISE

Code_Exercises/What_is_SYCL/source Configure your environment for using SYCL and compile a source file with the SYCL compiler.

For MOGON NHR, open: mod.hpc.uni-mainz.de
Login using the provided credentials. Open "Code Server", start a new session with the parameters ⇒

Then follow the instructions in the README.md file in the source directory.
https://github.com/AdaptiveCpp/syclacademy/blob/nhr-uds/Code_Exercises/Section_1_What_is_SYCL/README.md

- Use `ki-heprosycl` as account
- Use `A40` as partition
- Number of hours: 9
- Number of Tasks: 1
- CPUs per Task: 8
- Memory: 32
- Number of GPUs: 1